

halfedge：内置属性模块设计文档

src/builtin_attrs.rs

halfedge 库

2026-07-05

目录

1 模块定位	2
2 为什么需要 Newtype 包装?	2
3 API 表	3
3.1 Newtype 列表	3
3.2 注册函数	3
3.3 批量计算与收集	4
3.4 OBJ 属性感知	4
4 属性数据流	4
5 OBJ 属性感知 IO	4
5.1 解析: parse_obj_with_attrs	4
5.2 序列化: format_obj_with_attrs	5
6 批量安装: install_vertex_attrs	5
7 与 property.rs 的关系	6
8 退化守卫	6
9 使用示例	6
9.1 为 icosphere 计算顶点法向并写出 OBJ	6
9.2 从带属性的 OBJ 加载	7
9.3 批量安装自定义属性	7
10 选择态与边/面色	7
10.1 Newtype 列表	7
10.2 语义约定	8
10.3 边级 (twin 对) 同步	8
10.4 API 表	8

10.5 使用示例	9
11 可视化尺寸 / 粗细 / 透明度	10
11.1 Newtype 列表	10
11.2 语义约定	10
11.3 边级 (twin 对) 粗细同步	10
11.4 API 表	10
11.5 使用示例	11
12 测试覆盖	12

1 模块定位

`builtin_attrs.rs` 在 `property.rs` 泛型属性系统之上，为常见几何属性（顶点法向 / UV / 颜色 / 面法向）提供强类型 Newtype 包装与便捷函数，并实现 **OBJ 属性感知 IO**（`v/vt/vn` 解析与序列化）。

2 为什么需要 Newtype 包装？

`property.rs` 的属性系统基于 `TypeId` 区分属性类型：

属性键 = `TypeId::of::<T>()`

`TypeId` 由 Rust 编译器为每个 `'static` 类型自动分配，相同类型共享同一 ID。**问题：**若想为顶点同时附加 `[f64; 3]` 类型的「法向」与「颜色」，二者 `TypeId` 相同，会互相覆盖。

解决方案：用 Newtype 包装，让每个语义属性有独立类型：

```

1 #[derive(Debug, Clone, Copy, Default, PartialEq)]
2 pub struct VertexNormal(pub [f64; 3]);
3
4 #[derive(Debug, Clone, Copy, Default, PartialEq)]
5 pub struct VertexColor(pub [f64; 3]);
6
7 #[derive(Debug, Clone, Copy, Default, PartialEq)]
8 pub struct VertexUv(pub [f64; 2]);
9
10 #[derive(Debug, Clone, Copy, Default, PartialEq)]
11 pub struct FaceNormal(pub [f64; 3]);

```

`VertexNormal` 与 `VertexColor` 现在是不同类型，`TypeId::of::<VertexNormal>()` $\neq$ `TypeId::of::<VertexColor>()`，可以共存于同一 `MeshProperties` 容器。

3 API 表

3.1 Newtype 列表

Newtype	底层类型	语义
<code>VertexNormal([f64;3])</code>	<code>[f64;3]</code>	顶点法向 (单位向量)
<code>VertexColor([f64;3])</code>	<code>[f64;3]</code>	顶点颜色 RGB, 每通道 $\in [0, 1]$
<code>VertexUv([f64;2])</code>	<code>[f64;2]</code>	顶点 UV 纹理坐标
<code>FaceNormal([f64;3])</code>	<code>[f64;3]</code>	面法向 (单位向量)
<code>VertexSelected(bool)</code>	<code>bool</code>	顶点选择态
<code>HalfEdgeSelected(bool)</code>	<code>bool</code>	半边选择态
<code>FaceSelected(bool)</code>	<code>bool</code>	面选择态
<code>HalfEdgeColor([f64;3])</code>	<code>[f64;3]</code>	半边/边颜色 RGB
<code>FaceColor([f64;3])</code>	<code>[f64;3]</code>	面颜色 RGB
<code>VertexSize(f64)</code>	<code>f64</code>	顶点显示半径
<code>HalfEdgeThickness(f64)</code>	<code>f64</code>	半边粗细 (twin 同步)
<code>FaceOpacity(f64)</code>	<code>f64</code>	面透明度 $\in [0, 1]$

3.2 注册函数

注册函数	返回类型
<code>add_vertex_normal(&mut props)</code>	<code>PropHandle<VertexNormal></code>
<code>add_vertex_colors(&mut props)</code>	<code>PropHandle<VertexColor></code>
<code>add_vertex_uvs(&mut props)</code>	<code>PropHandle<VertexUv></code>
<code>add_face_normal(&mut props)</code>	<code>PropHandle<FaceNormal></code>
<code>add_vertex_selection(&mut props)</code>	<code>PropHandle<VertexSelected></code>
<code>add_halfedge_selection(&mut props)</code>	<code>PropHandle<HalfEdgeSelected></code>
<code>add_face_selection(&mut props)</code>	<code>PropHandle<FaceSelected></code>
<code>add_halfedge_colors(&mut props)</code>	<code>PropHandle<HalfEdgeColor></code>
<code>add_face_colors(&mut props)</code>	<code>PropHandle<FaceColor></code>
<code>add_vertex_sizes(&mut props)</code>	<code>PropHandle<VertexSize></code>
<code>add_halfedge_thickness(&mut props)</code>	<code>PropHandle<HalfEdgeThickness></code>
<code>add_face_opacity(&mut props)</code>	<code>PropHandle<FaceOpacity></code>

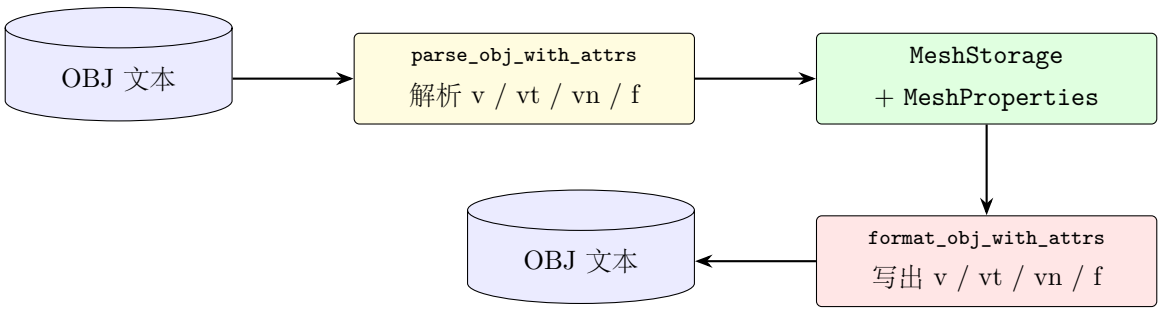
3.3 批量计算与收集

函数	语义
<code>populate_vertex_normal(...)</code>	用 <code>vertex_normal</code> 填充所有顶点
<code>populate_face_normal(...)</code>	用 <code>face_normal</code> 填充所有面
<code>collect_vertex_normal(...)</code>	按顶点顺序提取法向
<code>collect_vertex_uvs(...)</code>	按顶点顺序提取 UV
<code>collect_vertex_colors(...)</code>	按顶点顺序提取颜色

3.4 OBJ 属性感知

函数	语义
<code>parse_obj_with_attrs(text)</code>	解析 OBJ 并提取 <code>vt/vn</code>
<code>format_obj_with_attrs(...)</code>	序列化带 <code>vt/vn</code> 的 OBJ
<code>install_vertex_attrs(...)</code>	一次性注册并填充

4 属性数据流



5 OBJ 属性感知 IO

5.1 解析: `parse_obj_with_attrs`

输入: OBJ 文本, 含 `v / vt / vn / f` 行。输出: (`MeshStorage`, `MeshProperties`), 属性系统已注册 `VertexNormal` 与 `VertexUv` (仅在输入含对应行时)。

面行格式 支持 4 种:

- `f v1 v2 v3` ——仅顶点;
- `f v1/vt1 v2/vt2 v3/vt3` ——顶点 + UV;
- `f v1/vt1/vn1 v2/vt2/vn2 v3/vt3/vn3` ——顶点 + UV + 法向;
- `f v1//vn1 v2//vn2 v3//vn3` ——顶点 + 法向 (无 UV)。

属性映射 每个面顶点 (vt, vn) 索引 (1-based) 映射到顶点:

1. 收集所有 vt 行到 `texcoords: Vec<[f64;2]>`;
2. 收集所有 vn 行到 `normals: Vec<[f64;3]>`;
3. 遍历每个面顶点 (v_idx, vt_idx, vn_idx): `vert_normal[v_idx] = normals[vn_idx - 1]`;
4. 按顶点顺序注册属性并填充 `MeshProperties`。

同顶点多面引用 同一顶点可能被多个面引用, 后面的面覆盖前面的 (OBJ 标准允许同顶点在不同面有不同 UV/法向, 本实现按顶点而非按面顶点存储属性, 故取最后一次赋值)。

5.2 序列化: `format_obj_with_attrs`

输出顺序:

1. 所有 v 行 (顶点位置);
2. 所有 vt 行 (若 `uv_handle` 为 `Some`);
3. 所有 vn 行 (若 `normal_handle` 为 `Some`);
4. 所有 f 行, 按 v/vt/vn 格式输出 (依据已注册属性选择)。

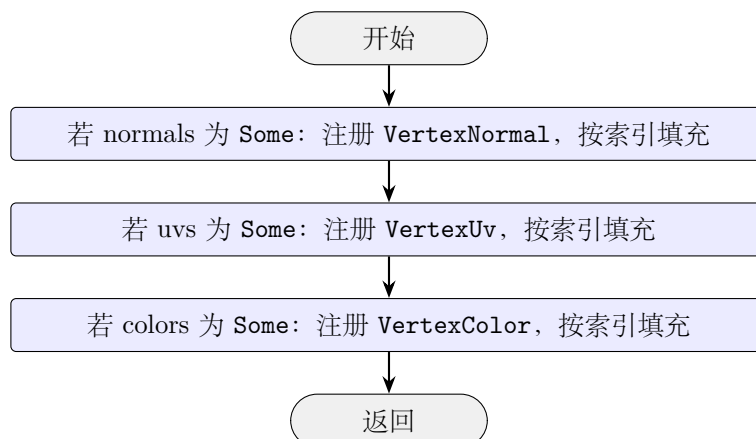
面行格式选择

- 同时有 UV 和法向: `f v/vt/vn v/vt/vn v/vt/vn`;
- 仅有 UV: `f v/vt v/vt v/vt`;
- 仅有法向: `f v//vn v//vn v//vn`;
- 都无: `f v v v`。

未设置属性的顶点写 0 占位 (`vt 0 0` 或 `vn 0 0 0`)。

6 批量安装: `install_vertex_attrs`

`install_vertex_attrs(props, normals, uvs, colors, vertex_ids)` 用于从外部数据 (如其他 IO 库解析结果) 一次性注册并填充属性:



部分输入：任一参数为 `None` 则跳过对应属性，`has_vertex_prop::<T>()` 对应返回 `false`。

7 与 `property.rs` 的关系

维度	<code>property.rs</code> (泛型)	<code>builtin_attrs.rs</code> (内置)
类型	任意 <code>T: 'static</code>	固定 4 种 Newtype
句柄	<code>PropertyHandle<T></code>	同左 (直接复用)
存储	<code>HashMap<TypeId, Box<dyn ErasedProperty>></code>	同左
注册	<code>add_vertex_prop::<T>()</code>	<code>add_vertex_normal(&mut props)</code>
读写	<code>get/set_vertex_prop</code>	同左
语义	无 (用户自定义)	法向 / UV / 颜色 / 面法向
IO 联动	无	<code>parse_obj_with_attrs</code> 等

`builtin_attrs` 完全建立在 `property.rs` 之上，不修改其底层存储。用户可同时使用两者：自定义泛型属性与内置属性互不冲突。

8 退化守卫

函数	退化场景	处理
<code>populate_vertex_normal</code>	顶点法向不可计算 (孤立 / 边界退化)	跳过该顶点
<code>populate_face_normal</code>	面法向不可计算 (共线)	跳过该面
<code>populate_face_normal</code>	空网格	安全返回 (无 panic)
<code>collect_vertex_normal</code>	顶点未设置法向	填 <code>[0, 0, 0]</code>
<code>collect_vertex_colors</code>	顶点未设置颜色	填 <code>[1, 1, 1]</code> (白)
<code>collect_vertex_uvs</code>	顶点未设置 UV	填 <code>[0, 0]</code>
<code>parse_obj_with_attrs</code>	OBJ 无 <code>vn</code> 行	不注册 <code>VertexNormal</code>
<code>parse_obj_with_attrs</code>	面索引越界	返回 <code>IndexOutOfRange</code> 错误
<code>parse_obj_with_attrs</code>	面顶点数 < 3	返回 <code>NotTriangular</code> 错误

9 使用示例

9.1 为 icosphere 计算顶点法向并写出 OBJ

```
1 use halfedge::*;
2 use halfedge::builtin_attrs::*;
3
4 let mesh = test_util::build_icosphere(2);
5 let mut props = MeshProperties::new();
6 let n_handle = add_vertex_normals(&mut props);
```

```

7 populate_vertex_normals(&mesh, &mut props, n_handle);
8
9 let obj = format_obj_with_attrs(&mesh, &props,
10     Some(n_handle), None);
11 // obj          v / vn / f      f      "f v//vn v//vn v//vn"

```

9.2 从带属性的 OBJ 加载

```

1 let text = std::fs::read_to_string("model.obj"?);
2 let (mesh, props) = parse_obj_with_attrs(&text)?;
3 // props          VertexNormal  VertexUv    OBJ    vn / vt

```

9.3 批量安装自定义属性

```

1 let mesh = build_icosphere(0);
2 let ids: Vec<VertexId> = mesh.vertex_ids().collect();
3 let mut props = MeshProperties::new();
4
5 let normals: Vec<[f64; 3]> = ids.iter().map(|_| [0.0, 1.0, 0.0]).collect();
6 let uvs: Vec<[f64; 2]> = ids.iter().enumerate()
7     .map(|(i, _)| [i as f64 * 0.1, 0.0])
8     .collect();
9
10 install_vertex_attrs(&mut props, Some(&normals), Some(&uvs), None, &ids);
11 assert!(props.has_vertex_prop:<VertexNormal>());
12 assert!(props.has_vertex_prop:<VertexUv>());
13 assert!(!props.has_vertex_prop:<VertexColor>());

```

10 选择态与边/面色

本节描述为顶点 / 半边 / 面三类元素提供的「选择态」与「边/面颜色」便捷 API。所有 API 都建立在 `MeshProperties` 之上，复用 Newtype 模式：

10.1 Newtype 列表

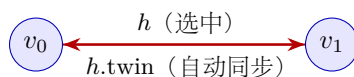
Newtype	底层类型	语义
<code>VertexSelected</code>	<code>bool</code>	顶点是否被选中
<code>HalfEdgeSelected</code>	<code>bool</code>	半边是否被选中
<code>FaceSelected</code>	<code>bool</code>	面是否被选中
<code>HalfEdgeColor</code>	<code>[f64;3]</code>	半边/边颜色 RGB
<code>FaceColor</code>	<code>[f64;3]</code>	面颜色 RGB

10.2 语义约定

- 未注册 $\equiv$ `false/None`: 未注册属性时, `is*_selected` 返回 `false`, `edge_color/face_color` 返回 `None`, 避免强制初始化。
- 按类型清理: `clear*_selection` 与 `clear_face_colors` 仅删除当前 `Newtype` 类型的属性, 不影响其他类型 (如 `VertexNormal`)。
- 懒迭代: `selected*_ids` 返回的迭代器按需过滤, 不预收集。

10.3 边级 (twin 对) 同步

无向边由两条互为 `twin` 的半边组成。从语义上, 边的选择/颜色应一致。`select_edge` / `deselect_edge` / `set_edge_color` 同时写入 `he` 与其 `twin`; `is_edge_selected` 任一半边被选中即返回 `true`; `selected_edge_ids` 按 canonical half ($he \leq twin$) 去重, 每个无向边只返回一次。



10.4 API 表

API	语义
顶点选择	
<code>select_vertex(props, h, id)</code>	选中顶点
<code>deselect_vertex(props, h, id)</code>	取消选中
<code>toggle_vertex_selection(props, h, id) → bool</code>	切换并返回新状态
<code>is_vertex_selected(props, h, id) → bool</code>	查询
<code>clear_vertex_selection(mesh, props, h)</code>	清空所有顶点选择
<code>selected_vertex_ids(mesh, props, h) → Iterator</code>	懒迭代选中顶点
<code>select_all_vertices(mesh, props, h)</code>	全选
<code>invert_vertex_selection(mesh, props, h)</code>	反选
<code>count_selected_vertices(mesh, props, h) → usize</code>	统计
半边选择	
<code>select_halfedge / deselect_halfedge</code>	同半边
<code>toggle_halfedge_selection</code>	同半边
<code>is_halfedge_selected</code>	同半边
<code>clear_halfedge_selection(mesh, props, h)</code>	清空
<code>selected_halfedge_ids(mesh, props, h) → Iterator</code>	懒迭代
面选择	
<code>select_face / deselect_face</code>	同面
<code>toggle_face_selection</code>	同面
<code>is_face_selected</code>	同面

API	语义
<code>clear_face_selection(mesh, props, h)</code>	清空
<code>selected_face_ids(mesh, props, h) → Iterator</code>	懒迭代
<code>select_all_faces(mesh, props, h)</code>	全选
<code>count_selected_faces(mesh, props, h) → usize</code>	统计
边级 (<i>twin</i> 同步)	
<code>select_edge(mesh, props, h, he)</code>	选中 he 与 twin
<code>deselect_edge(mesh, props, h, he)</code>	取消 he 与 twin
<code>toggle_edge_selection(mesh, props, h, he) → bool</code>	切换
<code>is_edge_selected(mesh, props, h, he) → bool</code>	任一 half 选中即 true
<code>selected_edge_ids(mesh, props, h) → Iterator</code>	canonical 去重
<code>set_edge_color(mesh, props, h, he, color)</code>	同步设置 he 与 twin 颜色
<code>edge_color(mesh, props, h, he) → Option<[f64;3]></code>	优先 he , 未设置则尝试 twin
面颜色	
<code>set_face_color(props, h, id, color)</code>	设置面颜色
<code>face_color(props, h, id) → Option<[f64;3]></code>	查询
<code>clear_face_colors(mesh, props, h)</code>	清空

10.5 使用示例

```

1 use halfedge::*;
2 let mesh = build_icosphere(1);
3 let mut props = MeshProperties::new();
4
5 //
6 let hv = add_vertex_selection(&mut props);
7 for v in mesh.vertex_ids() {
8     if is_boundary_vertex(&mesh, v) {
9         select_vertex(&mut props, hv, v);
10    }
11 }
12 let n_boundary = count_selected_vertices(&mesh, &props, hv);
13
14 //
15 let he = add_halfedge_selection(&mut props);
16 let some_edge = mesh.halfedge_ids().next().unwrap();
17 select_edge(&mesh, &mut props, he, some_edge);
18 assert!(is_edge_selected(&mesh, &props, he, some_edge));
19
20 //
21 let fc = add_face_colors(&mut props);
22 for (i, f) in mesh.face_ids().enumerate() {
23     let c = if i % 2 == 0 { [0.8, 0.2, 0.2] } else { [0.2, 0.8, 0.2] };
24     set_face_color(&mut props, fc, f, c);

```

11 可视化尺寸 / 粗细 / 透明度

本节描述为顶点 / 半边 / 面三类元素提供的「可视化尺寸」属性，用于控制渲染时的几何外观。所有 API 与颜色 API 形态一致，建立在 `MeshProperties` 之上。

11.1 Newtype 列表

Newtype	底层类型	语义
<code>VertexSize</code>	<code>f64</code>	顶点显示半径（单位与 <code>position</code> 一致），默认 0 = 不渲染标记
<code>HalfEdgeThickness</code>	<code>f64</code>	半边/边线宽（ <code>twin</code> 同步）
<code>FaceOpacity</code>	<code>f64</code>	面透明度 $\in [0, 1]$: 0 = 完全透明，1 = 完全不透明

11.2 语义约定

- **未注册 $\equiv$ None**: 未注册属性时, `vertex_size` / `edge_thickness` / `face_opacity` 返回 `None`, 避免强制初始化。
- **clamp 透明度**: `set_face_opacity` 与 `set_uniform_face_opacity` 自动将输入 clamp 到 $[0, 1]$, 杜绝越界。
- **按类型清理**: `clear_*` 仅删除当前 Newtype 类型的属性, 不影响其他类型。
- **批量统一**: `set_uniform_*` 一行代码为所有元素设置同一值。

11.3 边级 (twin 对) 粗细同步

与边颜色 API 一致, `set_edge_thickness` 同时写入 `he` 与其 `twin`; `edge_thickness` 优先读 `he`, 未设置则 fallback 到 `twin`; `set_uniform_edge_thickness` 仅处理 canonical half ($he \leq twin$), 避免重复设置。

若需非对称渲染（两条半边不同粗细），可使用 `set_halfedge_thickness` / `halfedge_thickness` 单半边版本。

11.4 API 表

API	语义
顶点大小 <code>set_vertex_size(props, h, id, size)</code>	设置顶点显示半径

API	语义
<code>vertex_size(props, h, id) → Option<f64></code>	查询
<code>clear_vertex_sizes(mesh, props, h)</code>	清空
<code>set_uniform_vertex_size(mesh, props, h, size)</code>	批量统一
边粗细 (<i>twin</i> 同步)	
<code>set_edge_thickness(mesh, props, h, he, t)</code>	同步设置 he 与 twin
<code>edge_thickness(mesh, props, h, he) → Option<f64></code>	优先 he , 未设置则尝试 twin
<code>set_halfedge_thickness(props, h, he, t)</code>	单半边设置 (不同步 twin)
<code>halfedge_thickness(props, h, he) → Option<f64></code>	单半边查询
<code>clear_halfedge_thickness(mesh, props, h)</code>	清空
<code>set_uniform_edge_thickness(mesh, props, h, t)</code>	批量统一 (canonical 去重)
面透明度	
<code>set_face_opacity(props, h, id, op)</code>	设置面透明度 (自动 clamp)
<code>face_opacity(props, h, id) → Option<f64></code>	查询
<code>clear_face_opacity(mesh, props, h)</code>	清空
<code>set_uniform_face_opacity(mesh, props, h, op)</code>	批量统一 (自动 clamp)

11.5 使用示例

```

1 use halfedge::*;
2 let mesh = build_icosphere(1);
3 let mut props = MeshProperties::new();
4
5 //
6 let hv = add_vertex_sizes(&mut props);
7 set_uniform_vertex_size(&mesh, &mut props, hv, 0.02);
8
9 let he = add_halfedge_thickness(&mut props);
10 set_uniform_edge_thickness(&mesh, &mut props, he, 0.005);
11
12 let hf = add_face_opacity(&mut props);
13 set_uniform_face_opacity(&mesh, &mut props, hf, 0.85);
14
15 //
16 let f0 = mesh.face_ids().next().unwrap();
17 set_face_opacity(&mut props, hf, f0, 0.5);
18 assert_eq!(face_opacity(&props, hf, f0), Some(0.5));
19
20 // clamp
21 set_face_opacity(&mut props, hf, f0, 1.5);
22 assert_eq!(face_opacity(&props, hf, f0), Some(1.0));

```

12 测试覆盖

测试名	验证点
基础属性	
vertex_normal_roundtrip	icosphere 顶点法向为单位向量
face_normal_roundtrip	所有面法向已写入
vertex_color_default_is_white	未设置颜色返回默认白
vertex_uv_set_and_get	UV 设置后可读取
install_vertex_attrs_handles_partial_input	仅传法向不注册 UV/Color
install_vertex_attrs_all_three	三种属性同时安装
newtype_default_is_zero	Newtype 默认值为零
missing_handle_returns_none	未注册句柄读取返回 None
populate_face_normal_skips_invalid	空网格不 panic
选择态与边/面色	
vertex_selection_basic	select / toggle / is / count
vertex_select_all_and_invert	全选 + 反选
vertex_clear_selection_keeps_registration	清空后类型注册保留
selected_vertex_ids_iterator_correct	懒迭代器返回选中顶点
face_selection_basic	面 select / select_all / clear
halfedge_selection_basic	半边 select / toggle / clear
edge_selection_syncs_twin_pair	选中半边时 twin 同步
selected_edge_ids_returns_canonical_only	canonical half 去重
edge_color_syncs_twin_pair	颜色同步到 twin
face_color_basic	面颜色 set/get/clear
selection_newtype_default_is_false	选择态默认 false
color_newtype_default_is_zero	颜色默认 [0,0,0]
unregistered_selection_safe_no_panic	未注册查询不 panic
unregistered_edge_safe_no_panic	未注册边查询不 panic
empty_mesh_selection_no_panic	空网格调用全选/反选不 panic
可视化尺寸 / 粗细 / 透明度	
vertex_size_basic	set / uniform / clear + 类型注册保留
vertex_size_default_is_zero	Newtype 默认值为 0
edge_thickness_syncs_twin_pair	twin 同步 + fallback 读取
edge_thickness_uniform_sets_all	批量统一设置所有边
halfedge_thickness_independent_set	单半边设置不影响 twin
face_opacity_basic	set / clamp / uniform / clear
face_opacity_default_is_zero	Newtype 默认值为 0
unregistered_size_thickness_opacity_safe	未注册查询不 panic
empty_mesh_size_thickness_opacity	空网格批量操作不 panic

`src/builtin_attrs.rs` 共 **33** 个单元测试，全库共 **708** 个。